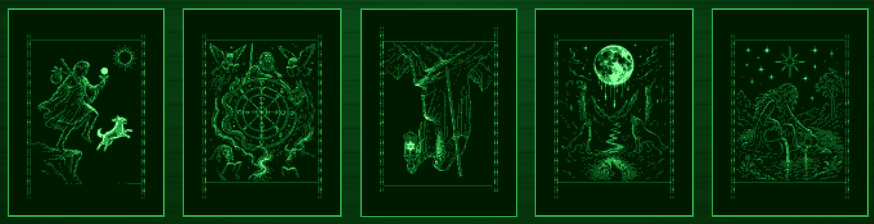
USER MANUAL · ENGLISH

THE NECRO-ORACLE

"Shuffle the deck. Name the repository. The dead will speak."



necro-oracle.vercel.app · github.com/FernandoLizana/necro-oracle · commit 5930ae6

NECRO>

Live: https://necro-oracle.vercel.app · **Source:** https://github.com/FernandoLizana/necro-oracle · **Demo video:** https://youtu.be/LqITM5tS1Oo

This manual describes commit `5930ae6` (main). The command syntax is taken from the terminal's own parser (`packages/ui-crt/src/command/parse.ts`) and help screen (`packages/ui-crt/src/render/help.ts`).

1. Getting started

1. Open **https://necro-oracle.vercel.app** in a desktop or mobile browser. No account or key is needed.
2. A short **boot sequence** plays ("NECROMANTIC SYSTEMS 1986"). Press any key to skip it.
3. The **landing menu** appears. It is laid out like a Game Boy cartridge menu: - **HUD bar** at the top: `NECRO-ORACLE`, three hearts and `PY2 CURSE: ACTIVE`. - **LEGACY REPOS**: three ready-to-read Python 2 repositories plus `> ENTER ANY REPO...`. - **PREVIEW**: the Seer's portrait, three card backs and a one-line comment from the oracle about the selected repo. - **Hint bar**: `▲▼ SELECT A:SUMMON B:DEMO START:HELP`.
4. Press **Enter** on the first repo. The oracle **summons** it and lays a **Fates** reading (three cards).
5. Read the cards, type `explain 2` (or `explain present`) to see the evidence, then type `resurrect` to raise the dead code. After a successful resurrect, click **LOVERS** (or type `lovers <file> js`) to bind the revived Python to a JavaScript client.

The `NECRO>` prompt at the bottom accepts typed commands. The **button row** under it offers a clickable version of every command. The `SOUL SOURCE` line tells you who pays for AI calls: **ORACLE** (the hosted server key) or **YOURS** (your own key).



The landing menu (▲▼ select · A summon · B demo · START help)

Landing menu controls

Key	Action
↑ / ↓	Move the selection (stops at the top and bottom, no wrap-around)
Enter or **A**	Activate: summon the selected repo and read `fates`. On `> ENTER ANY REPO...` it focuses the prompt instead
B or **D**	Start the offline demo
H	Show help

Keys pressed with Ctrl, Alt or Meta are never captured by the menu, so browser shortcuts keep working. The menu only reacts while it has focus, and anything typed into `NECRO>` goes to the prompt. Type `home` at any time to return to the menu.

The three legacy repos are `tomaes/pytarl` (DUNGEON · 4 FILES), `ArmstrongJ/robotadventure` (NOIR MYSTERY · 21 FILES) and `neubig/test-repo` (PY2 SPECIMEN · 240 FILES).

2. Commands

Commands are case-insensitive. Arguments are separated by spaces. An entry can be up to 200 characters. Anything the terminal doesn't recognize prints `Unknown command: … Type help for the grimoire of commands.`



help: the grimoire of commands

`summon <owner/repo[#ref|@sha]>`

Pins a public GitHub repository to an exact commit. Accepted forms:

- `summon psf/requests`, which uses the default branch
- `summon owner/repo#branch-or-tag`
- `summon owner/repo@<commit-sha>`
- `summon https://github.com/owner/repo`, a full URL (with or without the scheme)

Shortcut: typing just a repo reference on its own line (for example `tomaes/pytarl`) also summons it. The oracle replies with the pinned `owner/repo @ sha`, the file count and stars, and whether the snapshot came from cache. Private repositories can't be read.

`read [fates|necropsy|celtic]`

Lays a spread for the summoned repo. Bare `read` defaults to **necropsy**.

Spread	Cards	Positions
`fates`	3	Past · Present (the most severe affliction) · Future

Spread	Cards	Positions
`necropsy`	5	The Corpse · The Wound · The Shadow · The Hope · The Call (whether a resurrection ritual is available, and why)
`celtic`	10	Present · Crossing · Foundation · Recent Past · Crown · Near Future · Self · Environment · Hopes and Fears · Outcome

Each card shows its numeral, name, **orientation** (UPRIGHT = active problem, REVERSED = latent or partial risk), severity, a mystical line, the technical diagnosis and the top evidence items. The reading ends with a **VITALITY 0–100** gauge. A reading streams in phases: SUMMON, DOWNLOAD, EXTERNAL (registry/OSV), then READING.

The 12 diagnosis cards: XX Judgement (Python 2 runtime), XVI The Tower (dependencies), XV The Devil (security), XIII Death (dead code), X Wheel of Fortune (tests), IX The Hermit (docs), XII The Hanged Man (abandonment), XIV Temperance (CI/linting), XI Justice (licence/governance), XVIII The Moon (complexity), XVII The Star (strengths, always drawn) and XXI The World (overall health).

`explain <position|card>`

Shows the full evidence behind a card. Examples:

- `explain 2`: the card in position 2 of the current spread
- `explain past`, `explain present`, `explain future`: aliases of `explain 1`, `explain 2` and `explain 3`
- `explain judgement`, `explain hanged_man`: by card id

Valid card ids: `judgement tower devil death wheel hermit hanged_man temperance justice moon star world`.

`resurrect [<card-or-file>]`

Raises the dead code the cards named (Python 2 to Python 3). The **RESURRECT** button appears when the reading drew JUDGEMENT.

- `resurrect` resurrects every Python 2 candidate (at most 50 files)
- `resurrect judgement` resurrects by card id
- `resurrect src/legacy/tool.py` resurrects one file (any token containing `/` or ending in `.py` is treated as a path)

What happens:

1. The **Pre_Pass**: six deterministic fixers rewrite print statements, `except E, e:`, `<>`, long literals (`10L`), dict iteration methods and unicode/str usage.
2. If diagnostics remain, the **LLM pass** sends only those residual chunks to a model. It uses the FAST tier and escalates to STRONG for hard or large chunks.
3. A **validation gate** accepts only output that parses as Python 3 with zero errors and contains no surviving Python 2 construct. Otherwise the file is reported as unconverted, and the oracle never pretends.
4. The **Necronomicon ritual** animation plays (in full-motion mode), followed by the **diff**, the **before → after vitality** and a **downloadable patch**. On some screens the pane may not auto-scroll all the way to the end — scroll the output pane to reach the diff and **DOWNLOAD PATCH**.

`lovers <file> js`

Binds a resurrected Python 3 file to a **JavaScript client** (the "Frankenstein" ritual). Example: `lovers src/lich.py js` (`javascript` works as well as `js`). Judgement draws the `lovers-js` grimoire, so after a successful

`resurrect` the **LOVERS** chip appears in the button row and prefills `lovers <file>`. The pipeline runs PLAN → GENERATE → VALIDATE → BIND. The result is a split view and a **ZIP** containing the Python "heart" (a JSON-lines dispatcher), the JS "beloved" (a client that spawns the heart), a JSON contract schema and contract tests for both sides. If the pane does not auto-scroll to the end, scroll the output pane to reach the binding and **DOWNLOAD PAIR**. Preconditions: the reading must mark the file as bindable (Judgement with `lovers-js`), and the file must have up to 8 bindable top-level functions and be at most 32 KB. If not, the oracle refuses with a typed error and calls no model. Demo mode runs the same ritual on the Mock LLM with no network.

`tarot [question]`

Draws a **three-card destiny reading** (Past · Present · Future) from the full 22-card Major Arcana. Cards are always distinct, each independently upright or reversed, using a crypto-grade RNG. The question is optional free text (up to 200 characters) and is treated strictly as data. Examples: `tarot`, `tarot will my legacy code live?` The reveal is ceremonial: shuffle and cut, each card dealt and flipped with a Seer line, the **deterministic offline reading** typed paragraph by paragraph, then a **VERDICT**. With motion off it appears instantly. You don't need to summon a repo first. The model-backed FAST reading service (one call, ≤400 output tokens, its own daily ceiling) is built and tested but **not wired to a live `/api/tarot` route yet** — it is on the Roadmap; this command never spends AI credit today.

`key set | key status | key clear` (bring your own key)

See section 3.

`model <fast|strong> <id>`

Chooses which OpenRouter model to use for each tier **with your own key**, for this session only. Example: `model strong anthropic/claude-sonnet-4.5`. The id must look like `vendor/model` (optional leading `~`, optional `:tag`, up to 128 characters, no spaces). If no key is bound, the override is recorded but stays dormant: "the oracle keeps its own vessels".

`demo [name]`

Replays a recorded reading **with no network**. Names: `lich-py2` (a dead Python 2 repo), `mixed-ruins` (half-ruined) and `green-garden` (healthy). Bare `demo` lists the catalog. A demo **autoplays**: it shows the summary, lays the `fates` reading and ends with a seeded tarot draw. Every block is labelled `DEMO REPLAY · a recorded reading, no live summon.` Type `home` to leave the demo, or `summon` a real repo to replace it.

`motion [auto|on|off]`

Controls reveal and CRT animation. `auto` follows your OS "reduce motion" setting. Bare `motion` reports the current mode ("Motion is …"). The preference is remembered in this browser.

`sound [on|off]`

Turns the spectral hum, sound cues and the background theme on or off. Sound is **off by default** and only starts after a user gesture. Bare `sound` reports the current state, and `sound on` confirms with "Sound set to on." The preference is remembered in this browser.

`help`, `clear`, `home`

- `help` lists every command and keyboard shortcut.
- `clear` (or **Ctrl+L**) clears the screen but **keeps the reading** in memory, so `explain` and `resurrect` still work.
- `home` returns to the landing menu and leaves any demo replay.

Button row

`HELP · SUMMON · READ · DEMO · TAROT · CLEAR · MOTION · SOUND · KEY · MODEL`, plus context buttons: **RESURRECT** appears when the reading drew JUDGEMENT, and **LOVERS** appears when Judgement linked `lovers-js` and a resurrected bindable file is ready. Buttons that need an argument (SUMMON, MODEL, LOVERS) prefill the prompt (e.g. `summon`) and focus it so you can finish typing. The rest run immediately.

3. BYOK: bring your own OpenRouter key

By default the hosted oracle pays for AI calls (**SOUL SOURCE: ORACLE**) within its daily limits. To use your own OpenRouter account:

1. Type `key set` (or click **KEY**). The oracle answers *"Whisper your key to the void…"* and a **masked** input field opens.
2. Paste your OpenRouter key into that field and press **Enter**. Press **Escape** to cancel; the field is wiped and focus returns to the prompt.
3. The oracle confirms: *"The key is bound. SOUL SOURCE: YOURS; the last four runes are …"*.
4. Optional: choose models with `model fast <id>` and `model strong <id>`.
5. `key status` shows whether a key is bound, revealing only its **last 4 characters**.
6. `key clear` forgets the key immediately and cancels any request in flight. SOUL SOURCE returns to ORACLE.

How your key is protected

- **Never type the key on the command line.** If you type `key set sk-…`, the oracle intercepts it *before* it is echoed, stored in history or reported as an unknown command. It opens the masked field and warns you: *"Never speak your key aloud on the command line."*
- The key lives **only in this tab's memory**. It is not written to localStorage, sessionStorage, cookies or IndexedDB. Reloading the page or opening a new tab means no key.
- It is sent to the server in a single dedicated header, used for **that one request**, then dropped. It is never logged, cached, echoed in a response or included in a cache key.
- The upstream endpoint is fixed, so the key can only ever go to OpenRouter.
- If your key is rejected (invalid, or out of credit), you get a clear typed error. The oracle **never silently falls back** to the server key.
- With your own key, the server's daily spend ceilings don't apply to you, and you pay only for your own calls.

4. Server-mode limits (SOUL SOURCE: ORACLE)

To keep the free hosted oracle alive for everyone:

Limit	Value
Summons per IP per hour	20
Readings per IP per hour	10
Resurrections per IP per hour	5
Lovers bindings per IP per hour	5
Daily request ceiling (overall)	1000 by default
Daily server LLM budget / Lovers / tarot ceilings	1000 / 200 / 300 by default, each with its own counter
LLM calls per reading	12 by default (at most 2 per unit: FAST, then one escalated retry)
Repository size	50 MB compressed / 200 MB uncompressed
Files resurrected per ritual	up to 50 candidates

When a limit is reached you'll see `ERROR RATE_LIMITED` ("The gate is barred for a little while.") with a retry time. When the server's daily AI budget is spent, the oracle **degrades softly**: deterministic results still render, and the tarot reading uses its offline text. Binding your own key lifts the server AI ceilings for you.

5. Sound and music

All audio is **synthesized live** with the Web Audio API. There are no audio files.

- `sound on` enables a low spectral hum, short cues (summon, reveal) and an original looping chiptune in A minor (Am–F–C–G, square and triangle voices).
- Volume is capped by design. Sound never autoplays before you interact, and the ambient drone stays silent in reduced-motion mode.
- `sound off` silences everything immediately.

6. Demo mode and URL flags

Flag	Effect
`?boot=skip`	Skip the boot sequence and land directly on the menu
`?ritual=force`	Always play the full Necronomicon resurrection ritual, even with `?boot=skip` or in automated runs

Example: `https://necro-oracle.vercel.app/?boot=skip&ritual=force`

The demo (`demo lich-py2`, or **B** on the menu) is fully offline and deterministic, with the same cards and the same glitches every time. It's useful for presentations or when the network is unreliable.

7. Keyboard shortcuts

Key	Action
Enter	Run the typed command

Key	Action
↑ / ↓ (in the prompt)	Walk the command history
Ctrl+L	Clear the screen (the reading is kept)
Escape	Skip the current reveal or animation; cancel the masked key field
Ctrl+C	Abort the running summon or reading
↑ ↓ · Enter/A · B/D · H (landing)	Navigate, summon, demo, help
Tab	Normal browser focus order (never trapped)

8. Error states and troubleshooting

Errors are shown in two layers: a short mystical line, then the plain technical fact.

Code	What it means	What to do
`INVALID_REPO_URL`	The repo reference is malformed	Use `owner/repo` or a `github.com/owner/repo` URL
`REPO_NOT_FOUND`	No such public repo	Check the spelling
`REPO_PRIVATE`	The repo is private	Only public repos can be read
`REF_NOT_FOUND`	The branch, tag or sha doesn't exist	Omit the ref to read the default branch
`DOWNLOAD_TOO_LARGE` / `FILE_TOO_LARGE`	The repo or a file exceeds the size caps	Try a smaller repo, or resurrect a single file
`RATE_LIMITED`	A per-IP or daily limit was reached	Wait for the retry time, or bind your own key
`NETWORK_TIMEOUT`	Connection failed or timed out	Check your connection and retry, or use `demo`
`UPSTREAM_UNAVAILABLE`	GitHub or the AI provider is down, or your key is out of credit	Retry later, or top up your OpenRouter credit
`AUTH_INVALID`	Your OpenRouter key was rejected	`key clear`, then `key set` with a valid key
`VALIDATION_FAILED`	A reply failed schema validation, or a precondition wasn't met (e.g. `lovers` on a non-bindable file)	Re-run the reading, then retry
`PARSER_INIT_FAILED`	The code-parsing engine couldn't start on the server	Rare after the `619ebc4` grammar-shipping fix. Retry; if it persists, use `demo lich-py2` and report it
`CARD_NOT_FOUND`	Unknown card id in `explain`	Use one of the 12 ids listed above

Other tips:

- **"Unknown command"**: check the syntax with `help`. Commands take an exact number of arguments (e.g. `lovers <file> js` needs both).
- **Nothing to explain or resurrect**: summon and `read` a repo first.
- **Only one ritual at a time**: wait for the current one to finish, or press Ctrl+C.
- **No sound**: run `sound on` *and* interact with the page once, and check that the tab isn't muted.
- **Too much motion**: `motion off`, or enable your OS "reduce motion" setting and use `motion auto`.

- **Can't see the diff or download after resurrect / Lovers**: scroll the output pane. On some screens the result view does not auto-scroll all the way to the end; the diff and **DOWNLOAD PATCH** / **DOWNLOAD PAIR** are present just below.

9. FAQ

Does the oracle run my repository's code? No. Repository contents are only parsed (tree-sitter, manifest parsers) and treated as untrusted data. They are never executed, imported or evaluated.

Can the AI make up a diagnosis? No. Each card is drawn by a deterministic detector with evidence. The AI only rewrites residual code, and every output must pass validation gates.

Is my key stored? No. It's held in this tab's memory only and used per request. See section 3.

Does it open pull requests on my repo? No. Resurrection produces a diff and a downloadable patch that you apply yourself.

Which languages are supported? Diagnosis and resurrection target Python (2 to 3). The Lovers binding generates a JavaScript client for Python 3 code.

Does it work offline? The `demo` replays (reading, resurrect and Lovers on the Mock LLM) and the deterministic tarot reading do. Live summons, live resurrect and live Lovers need the network (and an OpenRouter key for the LLM passes).

Is it accessible? Yes. It's real DOM text with a single live region for screen readers, fully keyboard-operable, reduced-motion aware, axe-clean, and usable at 320 px width and 200 % zoom.

Where does the card art come from? It is a pixel-art treatment of the Rider–Waite–Smith deck (Pamela Colman Smith, 1909, public domain).

10. Local development (for judges who want to run it)

Requirements: **Node 22**, **pnpm 10** (the repo pins `pnpm@10.11.0` via `packageManager`).

```
git clone https://github.com/FernandoLizana/necro-oracle
cd necro-oracle
pnpm install
pnpm -r build            # builds every package (the web app imports their dist output)
pnpm dev                 # Next.js dev server for apps/web (http://localhost:3000)
```

No environment variables are required. Without keys the app runs on the deterministic **Mock** provider: readings render, resurrection runs the deterministic Pre_Pass, and tarot uses its offline reading. Optional server variables (see `docs/MODELS.md`): `OPENROUTER_API_KEY`, `NECRO_LLM_MODEL_FAST`, `NECRO_LLM_MODEL_STRONG`, `NECRO_LLM_MODEL_FALLBACKS`, `GITHUB_TOKEN`, `UPSTASH_REDIS_REST_URL` / `_TOKEN`, `NECRO_PUBLIC_URL` and the `NECRO_*_CEILING` limits. Never commit a `.env*` file; a Kiro hook blocks agent writes to them.

Quality gates:

```
pnpm test                                  # Vitest: unit + property-based tests
pnpm typecheck                             # tsc project references + web
pnpm lint                                  # ESLint, --max-warnings=0
pnpm -r build                              # all packages + web
node scripts/check-web-budget.mjs          # 250 KB gz budget, chunk split, leak markers
pnpm --filter @necro/web check:wasm-trace  # tree-sitter WASM packaged into functions
pnpm --filter @necro/web exec playwright install   # once
pnpm e2e                                   # Playwright e2e + axe
pnpm smoke:llm                             # LLM plumbing, offline by default
pnpm demo:bundle                           # regenerate the offline demo bundle
```

Status at `5930ae6`: 3,091 unit and property tests passing (301 files), 52 Playwright e2e passing, typecheck clean, lint at 0 warnings, and build, budget and symlink-aware WASM-trace checks all green. Production on https://necro-oracle.vercel.app runs Node 22.x; live reading, resurrect and Lovers were verified on 2026-11-09 against `tomaes/pytarl`: `resurrect pytarl.py` raised vitality **0 → 12** with **no LLM cost** (deterministic Pre_Pass); `lovers pytarl.py javascript` bound **7 functions** for **6,418 tokens** (**0.0842 USD**), after an earlier Lovers run on the same file via `anthropic/claude-sonnet-4.5` for **0.019 USD**.